

D2.1 Unit description

Transportation problems; allocation (assignment) problems; the travelling salesman; game theory; further linear programming, dynamic programming; flows in networks

D2.2 Assessment information

Prerequisites and Preamble

Prerequisites

Students should be familiar with the material in, and the glossary attached to, the D1 specification.

Preamble

Students should be familiar with the terms defined in the glossary attached to this specification. Students should show clearly how an algorithm has been applied.

Examination

The examination will consist of one 1½ hour paper. The paper will contain about seven questions with varying mark allocations per question which will be stated on the paper. All questions may be attempted.

Calculators

Students are expected to have available a calculator with at least the following keys: $+$, $-$, $\times$, $\div$, π , x^2 , $\sqrt{x}$, $\frac{1}{x}$, x^y and memory. Calculators with a facility for symbolic algebra, differentiation and/or integration are not permitted.

Formulae

Students are expected to know any other formulae which might be required by the specification and which are not included in the booklet, *Mathematical Formulae including Statistical Formulae and Tables*, which will be provided for use with the paper. Questions will be set in SI units and other units in common usage.

1 **Transportation problems**

What students need to learn:

The north-west corner method for finding an initial basic feasible solution.

Problems will be restricted to a maximum of four sources and four destinations. The idea of dummy locations and degeneracy are required.

Use of the stepping-stone method for obtaining an improved solution. Improvement indices.

Students should identify a specific entering cell and a specific exiting cell.

Formulation as a linear programming problem.

2 **Allocation (assignment) problems**

What students need to learn:

Cost matrix reduction.

Students should reduce rows first.

The size of the cost matrix will be at most 5×5 .

Use of the Hungarian algorithm to find least cost allocation.

The idea of a dummy location is required.

Modification of method to deal with a maximum profit allocation.

Formulation as a linear programming problem.

3 The travelling salesman problem

What students need to learn:

The practical and classical problems. The classical problem for complete graphs satisfying the triangle inequality.

Determination of upper and lower bounds using minimum spanning tree methods.

The nearest neighbour algorithm.

The use of short cuts to improve upper bound is included.

The conversion of a network into a complete network of shortest 'distances' is included.

4 Further linear programming

What students need to learn:

Formulation of problems as linear programs.

The Simplex algorithm and tableau for maximising problems.

The use and meaning of slack variables.

Problems will be restricted to those with a maximum of four variables and four constraints, in addition to non-negativity conditions.

5 Game theory

What students need to learn:

Two person zero-sum games and the pay-off matrix. Identification of play safe strategies and stable solutions (saddle points).

Reduction of pay-off matrices using dominance arguments. Optimal mixed strategies for a game with no stable solution.

Conversion of 3×2 and 3×3 games to linear programming problems.

Use of graphical methods for $2 \times n$ or $n \times 2$ games where $n = 1, 2$ or 3 .

6 Flows in networks

What students need to learn:

Algorithm for finding a maximum flow.

Cuts and their capacity.

Use of the max flow — min cut theorem to verify that a flow is a maximum flow.

Multiple sources and sinks.

Vertex restrictions are not required. Only networks with directed arcs will be considered. Only problems with upper capacities will be set.

7 Dynamic programming

What students need to learn:

Principles of dynamic programming. Bellman's principle of optimality.

Stage variables and State variables. Use of tabulation to solve maximum, minimum, minimax or maximin problems.

Both network and table formats are required.

Glossary for D2

1 Transportation problems

In the **north-west corner method**, the upper left-hand cell is considered first and as many units as possible sent by this route.

The **stepping stone method** is an iterative procedure for moving from an initial feasible solution to an optimal solution.

Degeneracy occurs in a transportation problem, with m rows and n columns, when the number of occupied cells is less than $(m + n - 1)$.

In the transportation problem:

The **shadow costs** R_i , for the i th row, and K_j , for the j th column, are obtained by solving $R_i + K_j = C_{ij}$ for **occupied cells**, taking $R_1 = 0$ arbitrarily.

The **improvement index** I_{ij} for an **unoccupied cell** is defined by $I_{ij} = C_{ij} - R_i - K_j$.

3 The travelling salesman problem

The **travelling salesman problem** is ‘find a route of minimum length which visits every vertex in an undirected network’. In the ‘**classical**’ problem, each vertex is visited once only. In the ‘**practical**’ problem, a vertex may be revisited.

For three vertices A , B and C , the **triangular inequality** is ‘length $AB \leq$ length $AC +$ length CB ’.

A **walk** in a network is a finite sequence of edges such that the end vertex of one edge is the start vertex of the next.

A walk which visits every vertex, returning to its starting vertex, is called a **tour**.

4 Further linear programming

The **simplex tableau** for the linear programming problem:

$$\begin{array}{ll} \text{Maximise} & P = 14x + 12y + 13z, \\ \text{Subject to} & 4x + 5y + 3z \leq 16, \\ & 5x + 4y + 6z \leq 24, \end{array}$$

will be written as

Basic variable	x	y	z	r	s	Value
r	4	5	3	1	0	16
s	5	4	6	0	1	24
P	-14	-12	-13	0	0	0

where r and s are slack variables.

5 Game theory

A **two-person game** is one in which only two parties can play.

A **zero-sum** game is one in which the sum of the losses for one player is equal to the sum of the gains for the other player.

6 Flows in networks

A **cut**, in a network with source S and sink T , is a set of arcs (edges) whose removal separates the network into two parts X and Y , where X contains at least S and Y contains at least T . The **capacity of a cut** is the sum of the capacities of those arcs in the cut which are directed from X to Y .

If a network has several sources $S_1, S_2, \dots$, then these can be connected to a single **supersource** S . The capacity of the edge joining S to S_1 is the sum of the capacities of the edges leaving S_1 .

If a network has several sinks $T_1, T_2, \dots$, then these can be connected to a **supersink** T . The capacity of the edge joining T_1 to T is the sum of the capacities of the edges entering T_1 .

7 Dynamic programming

Bellman's principle for dynamic programming is 'Any part of an optimal path is optimal.'

The **minimax route** is the one in which the maximum length of the arcs used is as small as possible.

The **maximin route** is the one in which the minimum length of the arcs used is as large as possible.